Ex. No. 7 & 8	Full Stack Web App using React and Node.js
Date of Exercise	19/10/2026
GitHub Host Link	https://gitstats-khaki.vercel.app/
Source Code	https://github.com/TheJebster/GitStats

Aim

To build GitStats, a full stack web application with a React front end and a Node.js (Express) server that reads live data from the GitHub REST API and shows the health of any public repository as a dashboard.

Description

GitStats is split into a client and a server. The React client has a home page with a search box and a dashboard page for one repository. It never talks to GitHub directly: a small fetch wrapper calls the server, and React Router decides which page to show.

The Express server exposes three JSON routes (/api/status, /api/top-repos and /api/repos/:owner/:repo). It validates the owner and repository names, calls the GitHub API, keeps the results in an in-memory cache for 10 minutes and works out a 0 to 100 health score from commit activity (35 points), issue response time (35 points) and pull request merge rate (30 points). Wrong input gives a 400 response, unknown routes give 404 and unexpected errors give 500. The app is hosted on Vercel.

Procedure:

1. Create the GitStats project with a client folder (React with Vite) and a server folder (Express), then run npm install.
2. In server/app.js create the Express app, enable cors and JSON parsing and add an API router.
3. Add patterns to check the owner and repository names and return a 400 error for invalid ones.
4. Add the routes GET /api/status, GET /api/top-repos and GET /api/repos/:owner/:repo, each wrapped in an async error handler.
5. Add a 404 handler for unknown API routes and an error handler that sends JSON with the right status code.
6. Write server/cache.js, a small cache with an expiry time that also shares one GitHub call between identical requests.
7. Write server/health.js with calculateHealth(), which returns the score, grade and breakdown.
8. Write server/index.js to serve the built React app and listen on port 5000.

9. In the client, wrap App in BrowserRouter (main.jsx), define the routes in App.jsx, and write api.js and parseRepo.js.
10. Run npm run dev to test locally, then deploy to Vercel, where vercel.json sends /api requests to the Express app.
11. Open the live app, search for a repository and check the dashboard and the API responses.

Program

Client (React)

client/src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App.jsx';
import './styles.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter future={{ v7_startTransition: true, v7_relativeSplatPath: true }}>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

client/src/App.jsx

```
import { Link, Route, Routes } from 'react-router-dom';
import Home from './pages/Home.jsx';
import Dashboard from './pages/Dashboard.jsx';

function Logo() {
  return (
    <svg className="brand-mark" viewBox="0 0 32 32" width="24" height="24" aria-
hidden="true">
      <rect width="32" height="32" rx="4" fill="#101010" stroke="#3d3a39" />
      <path d="M8 21V11M16 24V8M24 19v-6" stroke="#eeeeee" strokeWidth="3"
strokeLinecap="round" />
      <path d="M24 19v-6" stroke="#ee6018" strokeWidth="3" strokeLinecap="round" />
    </svg>
  );
}

function Header() {
  return (
    <header className="site-header">
      <div className="header-inner">
        <Link to="/" className="brand" aria-label="GitStats home">
          <Logo />
          <span className="brand-name">
            Git<span className="brand-accent">Stats</span>
          </span>
        </Link>
      </div>
    </header>
  );
}

function NotFound() {
```

```
  return (
    <div className="dashboard narrow">
      <div className="error-state">
        <span className="error-icon">404</span>
        <h3>Page not found</h3>
        <p>The page you were looking for does not exist.</p>
        <Link className="button" to="/">Back to search</Link>
      </div>
    </div>
  );
}

export default function App() {
  return (
    <div className="app-shell">
      <a className="skip-link" href="#main-content">Skip to content</a>
      <Header />

      <main id="main-content">
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/repo/:owner/:repo" element={<Dashboard />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
      </main>

      <footer className="site-footer">
        <div className="footer-inner">
          <span>GitStats — live repository health from the GitHub REST API.</span>
          <span className="footer-muted">Results are cached for 10 minutes on the
server.</span>
        </div>
      </footer>
    </div>
  );
}
```

client/src/api.js

```
const BASE_URL = import.meta.env.VITE_API_URL || '';

function makeError(message, code, extra = {}) {
  const error = new Error(message);
  error.code = code;
  Object.assign(error, extra);
  return error;
}

async function request(path) {
  let response;

  try {
    response = await fetch(`${BASE_URL}${path}`, {
      headers: { Accept: 'application/json' }
    });
  } catch {
    throw makeError('Could not reach the GitStats server. Is it running?',
'NETWORK_ERROR');
  }

  const data = await response.json().catch(() => ({}));
```

```
  if (!response.ok) {
    throw makeError(data.error || 'The request failed.', data.code ||
'REQUEST_FAILED', {
      status: response.status,
      rateLimit: data.rateLimit || null
    });
  }

  return data;
}

export function getTopRepos(sort = 'stars') {
  return request(`/api/top-repos?sort=${encodeURIComponent(sort)}`);
}

export function getRepoStats(owner, repo) {
  return request(`/api/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}
`);
}
```

client/src/parseRepo.js

```
/**
 * Accepts "facebook/react", "https://github.com/facebook/react",
 * "github.com/facebook/react/tree/main", or "git@github.com:facebook/react.git".
 */
export function parseRepoInput(raw) {
  if (typeof raw !== 'string') return null;

  let value = raw.trim();
  if (!value) return null;

  value = value
    .replace(/^git@github\.com:/i, '')
    .replace(/^https?:\/\/(www\.)?github\.com\//i, '')
    .replace(/^github\.com\//i, '')
    .split(/[?#]/)[0]
    .replace(/^\/+|\/+$/g, '')
    .replace(/\.git$/i, '');

  const parts = value.split('/').filter(Boolean);
  if (parts.length < 2) return null;

  const [owner, repo] = parts;

  if (!/^[A-Za-z0-9](?:[A-Za-z0-9-]{0,38})$/.test(owner)) return null;
  if (!/^[A-Za-z0-9._-]{1,100}$/.test(repo) || repo === '.' || repo === '..') return
null;

  return { owner, repo };
}
```

Server (Node.js and Express)

server/app.js

```
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import express from 'express';
```

```
import cors from 'cors';
import dotenv from 'dotenv';

import { GithubError, hasToken, getRateLimits, getRateLimitOverview } from
'./github.js';
import { getRepoStats, getTopRepos } from './stats.js';
import { withCache } from './cache.js';

const __dirname = path.dirname(fileURLToPath(import.meta.url));

// The .env file lives at the workspace root, not next to this file.
dotenv.config({ path: path.resolve(__dirname, '../.env') });
dotenv.config();

const app = express();
const CLIENT_ORIGIN = process.env.CLIENT_ORIGIN;

const OWNER_PATTERN = /^[A-Za-z0-9](?:[A-Za-z0-9-]{0,38})$/;
const REPO_PATTERN = /^[A-Za-z0-9._-]{1,100}$/;

app.disable('x-powered-by');
if (CLIENT_ORIGIN) {
  app.use(cors({ origin: CLIENT_ORIGIN }));
} else {
  app.use(cors());
}
app.use(express.json({ limit: '10kb' }));

function asyncHandler(handler) {
  return (req, res, next) => Promise.resolve(handler(req, res, next)).catch(next);
}

function validateRepoParams(owner, repo) {
  if (!OWNER_PATTERN.test(owner) || !REPO_PATTERN.test(repo) || repo === '.' || repo
=== '..') {
    throw new GithubError('Invalid repository name. Use the form owner/repository.', {
      status: 400,
      code: 'INVALID_REPO'
    });
  }
}

const apiRouter = express.Router();

apiRouter.get('/status', asyncHandler(async (req, res) => {
  let limits = {
    core: getRateLimits().core || null,
    search: getRateLimits().search || null
  };

  try {
    limits = await withCache('rate-limit-overview', 60_000, getRateLimitOverview);
  } catch {
    // GitHub may be unreachable. The last known headers are still useful.
  }

  res.json({
    tokenConfigured: hasToken(),
```

```
    rateLimit: limits.core,
    searchRateLimit: limits.search
  });
}));

apiRouter.get('/top-repos', asyncHandler(async (req, res) => {
  const sort = req.query.sort === 'rising' ? 'rising' : 'stars';
  const data = await getTopRepos(sort);
  res.json(data);
}));

apiRouter.get('/repos/:owner/:repo', asyncHandler(async (req, res) => {
  const { owner, repo } = req.params;
  validateRepoParams(owner, repo);

  const data = await getRepoStats(owner, repo);
  res.json(data);
}));

apiRouter.use((req, res) => {
  res.status(404).json({ error: 'API route not found.', code: 'API_NOT_FOUND' });
});

// Support both /api prefix and root mounting (for serverless environments)
app.use('/api', apiRouter);

app.use((error, req, res, next) => {
  if (res.headersSent) return next(error);

  if (error instanceof GithubError) {
    return res.status(error.status).json({
      error: error.message,
      code: error.code,
      rateLimit: error.rateLimit || null
    });
  }

  console.error('[gitstats]', error);
  return res.status(500).json({
    error: 'Something went wrong on the server.',
    code: 'INTERNAL_ERROR'
  });
});

export default app;
```

server/index.js

```
import fs from 'node:fs';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import express from 'express';
import app from './app.js';
import { hasToken } from './github.js';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const PORT = Number(process.env.PORT) || 5000;

// Serve the built React app when it exists (npm run build && npm start).
```

```
const clientDist = path.resolve(__dirname, '../client/dist');

if (fs.existsSync(path.join(clientDist, 'index.html'))) {
  app.use(express.static(clientDist));

  app.get('*', (req, res) => {
    res.sendFile(path.join(clientDist, 'index.html'));
  });
}

app.listen(PORT, () => {
  console.log(`GitStats API listening on http://localhost:${PORT}`);
  console.log(hasToken() ? 'GitHub token: configured' : 'GitHub token: not set (60
requests/hour)');
});
```

server/cache.js

```
/**
 * Tiny in-memory cache with a TTL.
 *
 * It also remembers in-flight promises, so two identical requests arriving at
 * the same time result in one GitHub call instead of two.
 */
const store = new Map();
const pending = new Map();

export async function withCache(key, ttlMs, loader) {
  const hit = store.get(key);

  if (hit) {
    if (hit.expiresAt > Date.now()) {
      return hit.value;
    }
    store.delete(key);
  }

  if (pending.has(key)) {
    return pending.get(key);
  }

  const promise = loader()
    .then((value) => {
      store.set(key, { value, expiresAt: Date.now() + ttlMs });
      return value;
    })
    .finally(() => {
      pending.delete(key);
    });

  pending.set(key, promise);
  return promise;
}

export function clearCache() {
  store.clear();
  pending.clear();
}

export function cacheStats() {
```

```
  return { entries: store.size, pending: pending.size };
}
```

server/health.js

```
const DAY = 24 * 60 * 60 * 1000;

export function median(values) {
  if (!values.length) return null;
  const sorted = [...values].sort((a, b) => a - b);
  const middle = Math.floor(sorted.length / 2);
  return sorted.length % 2 === 0
    ? (sorted[middle - 1] + sorted[middle]) / 2
    : sorted[middle];
}

function clamp(value, min, max) {
  return Math.min(max, Math.max(min, value));
}

function roundOne(value) {
  return Math.round(value * 10) / 10;
}

function gradeFor(score) {
  if (score >= 85) return { grade: 'Excellent', tone: 'excellent' };
  if (score >= 70) return { grade: 'Good', tone: 'good' };
  if (score >= 50) return { grade: 'Fair', tone: 'fair' };
  return { grade: 'Needs attention', tone: 'poor' };
}

/**
 * GitStats health score (0-100)
 *
 *   35 pts  commit activity   volume + consistency over the last 30 days
 *   35 pts  issue response    median time to close recently closed issues
 *   30 pts  pull requests     merge rate of recently closed PRs
 *
 * Components with no recent data receive neutral half-credit so a quiet but
 * healthy repository is not punished as if it were abandoned.
 */
export function calculateHealth({
  commits30d = 0,
  activeDays = 0,
  medianCloseDays = null,
  mergedPRs = 0,
  closedPRs = 0,
  issueSample = 0,
  prCapped = false
} = {}) {
  const commitPoints = roundOne(
    clamp(20 * Math.min(1, commits30d / 40) + 15 * Math.min(1, activeDays / 15), 0,
35)
  );

  const hasIssueData = medianCloseDays !== null && Number.isFinite(medianCloseDays);
  const issuePoints = hasIssueData
    ? roundOne(clamp(35 * (1 - Math.log10(1 + medianCloseDays) / Math.log10(31)), 0,
35))
    : 17.5;
```

```
  const hasPullData = closedPRs > 0;
  const prPoints = hasPullData ? roundOne(clamp(30 * (mergedPRs / closedPRs), 0,
30)) : 15;

  const score = Math.round(clamp(commitPoints + issuePoints + prPoints, 0, 100));
  const { grade, tone } = gradeFor(score);

  const formatDays = (days) => {
    if (days === null) return 'no recent data';
    if (days < 1) return 'under a day';
    if (days < 30) {
      const rounded = Math.round(days);
      return `${rounded} ${rounded === 1 ? 'day' : 'days'}`;
    }
    return `${(days / 30.44).toFixed(1)} months`;
  };

  return {
    score,
    grade,
    tone,
    breakdown: [
      {
        key: 'commits',
        label: 'Commit activity',
        points: commitPoints,
        max: 35,
        detail: `${commits30d}${commits30d >= 100 ? '+' : ''} commits on ${activeDays}
active ${activeDays === 1 ? 'day' : 'days'} in the last 30 days.`
      },
      {
        key: 'issues',
        label: 'Issue responsiveness',
        points: issuePoints,
        max: 35,
        detail: hasIssueData
          ? `Median close time of ${formatDays(medianCloseDays)} across ${issueSample}
recently closed ${issueSample === 1 ? 'issue' : 'issues'}.`
          : 'No recently closed issues to sample, so a neutral score is applied.'
      },
      {
        key: 'pulls',
        label: 'Pull request flow',
        points: prPoints,
        max: 30,
        detail: hasPullData
          ? `${mergedPRs} of ${closedPRs} recently closed pull requests merged$
{prCapped ? ' (latest 100 sampled)' : ''}.`
          : 'No recently closed pull requests to sample, so a neutral score is
applied.'
      }
    ]
  };
}

export { DAY };
```

Output

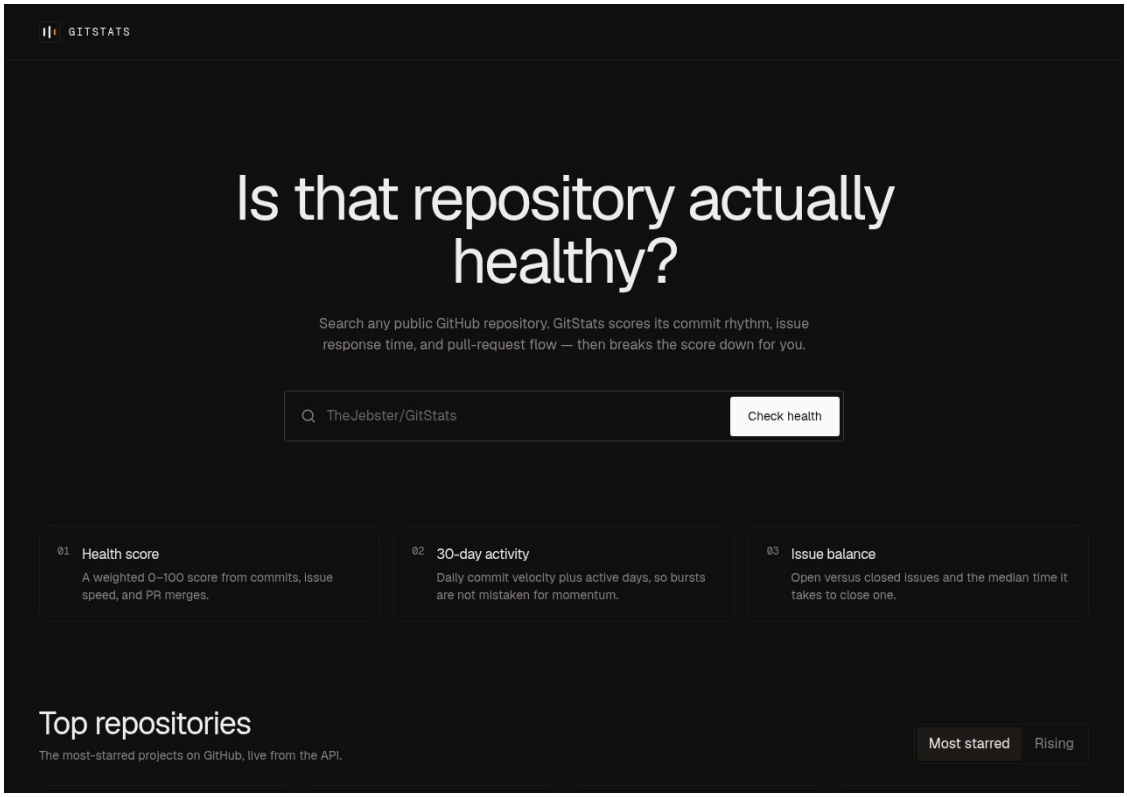


Fig. 7.1 GitStats home page with the repository search box



Fig. 7.2 Dashboard for facebook/react with health score and charts

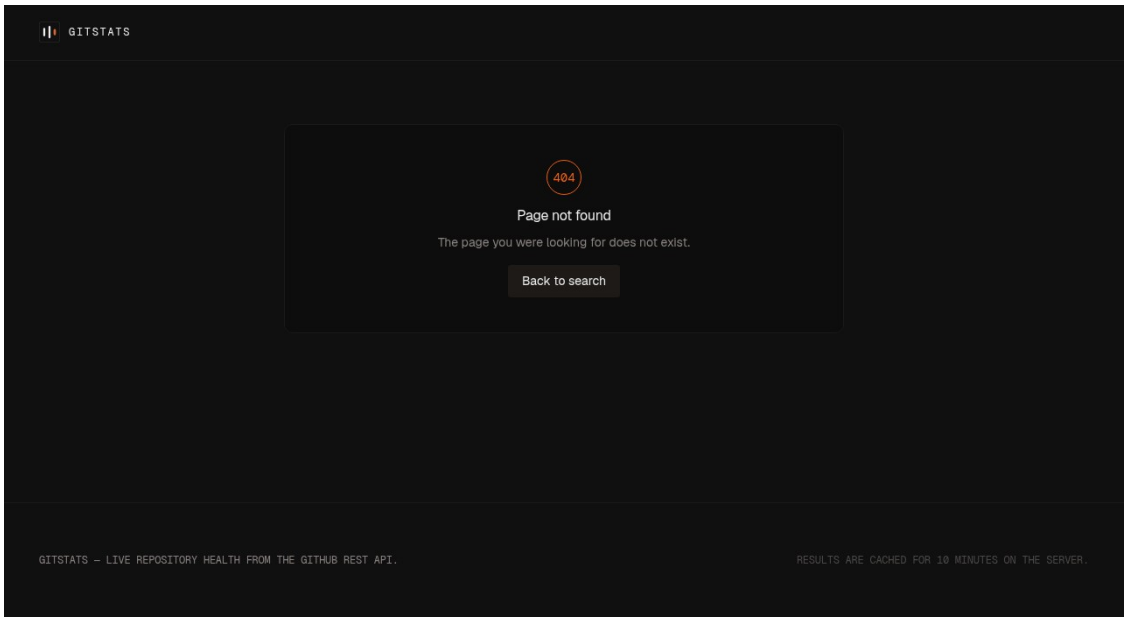


Fig. 7.3 Page not found screen for an unknown route

```
GET /api/repos/TheJebster/GitStats  (shortened)

{
  "repo": {
    "fullName": "TheJebster/GitStats",
    "stars": 0,
    "forks": 0,
    "watchers": 0,
    "language": "JavaScript",
    "pushedAt": "2026-11-01T17:27:31Z"
  },
  "health": {
    "score": 39,
    "grade": "Needs attention",
    "breakdown": [
      {
        "key": "commits",
        "points": 6,
        "max": 35
      },
      {
        "key": "issues",
        "points": 17.5,
        "max": 35
      },
      {
        "key": "pulls",
        "points": 15,
        "max": 30
      }
    ]
  },
  "activity": {
    "days": "[ ... ]",
    "total": 6,
    "activeDays": 3
  },
  "issues": {
    "open": 0,
    "closed": 0,
    "total": 0
  },
  "pullRequests": {
    "merged": 0,
    "closed": 0
  },
  "languages": [
    {
      "name": "JavaScript",
      "percent": 65
    },
    {
      "name": "CSS",
      "percent": 33.9
    },
    {
      "name": "HTML",
      "percent": 1.2
    }
  ]
}
```

Fig. 8.1 JSON returned by GET /api/repos/TheJebster/GitStats (shortened)

```
GET /api/status
HTTP 200
{
  "tokenConfigured": true,
  "rateLimit": {
    "limit": 5000,
    "remaining": 4981,
    "resetAt": "2026-11-02T09:50:30.000Z"
  },
  "searchRateLimit": {
    "limit": 30,
    "remaining": 30,
    "resetAt": "2026-11-02T09:23:48.000Z"
  }
}

GET /api/unknown
HTTP 404
{
  "error": "API route not found.",
  "code": "API_NOT_FOUND"
}

GET /api/repos/bad!!/x
HTTP 400
{
  "error": "Invalid repository name. Use the form owner/repository.",
  "code": "INVALID_REPO",
  "rateLimit": null
}
```

Fig. 8.2 Status route, unknown route (404) and invalid name (400) responses

Result

GitStats was built and deployed. The React client and the Node.js server worked together: the dashboard showed the health score, charts and statistics for a repository, and the server returned correct JSON and status codes for valid and invalid requests.